

54th CIRP Conference on Manufacturing Systems

Separating Entangled Workpieces in Random Bin Picking using Deep Reinforcement Learning

Marius Moosmann^{*a}, Marco Kulig^a, Felix Spenrath^a, Manuel Mönnig^a, Simon Roggendorf^b, Oliver Petrovic^b, Richard Bormann^a, Marco F. Huber^{a,c}

^aFraunhofer IPA, Nobelstraße 12, 70569 Stuttgart, Germany

^bLaboratory for Machine Tools and Production Engineering (WZL) of RWTH Aachen University, Steinbachstr. 19, 52074 Aachen, Germany

^cInstitute of Industrial Manufacturing and Management IFF, University of Stuttgart, Allmandring 35, 70569 Stuttgart, Germany

^{*} Corresponding author. Tel.: +49 711 970-3813; fax: +49 711 970-1008. E-mail address: marius.moosmann@ipa.fraunhofer.de

Abstract

Entangled workpiece situations often occur in random bin picking of chaotically stored objects and are a common source of problem in the bin picking process. Previous methods for averting this problem, such as randomly shaking the gripper over the bin, lead to decreasing production efficiency and an increase in cycle time. A promising new strategy uses supervised learning and deep neural networks to learn the separation. However, this approach requires a large amount of labeled data. To overcome this issue, this paper proposes a deep reinforcement learning approach to separate entangled workpieces and to minimize the setup effort.

© 2021 The Authors. Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the 54th CIRP Conference on Manufacturing System

Keywords: bin picking; deep reinforcement learning; entanglement; grasping; machine learning

1. Introduction

Workpieces are usually bulk goods and thus stored chaotically in bins in production environments. This provides a simple solution for storage and transport. For further processing operations the workpieces are picked out of the bin, which is difficult in case of entangled workpieces caused by complex shapes [1] [2]. These entangled components lead to a loss in robustness of the production process and therefore increase cycle times. To identify these situations Moosmann et al. [3] developed a method for detecting entangled workpiece situations. Furthermore a first separation strategy is described by Moosmann et al. [4], which is based on a supervised learning approach. One disadvantage of this method is the high expenditure of time needed for labeling training data. Therefore, this paper addresses a new deep reinforcement learning method for separation in order to decrease training time and setup effort.

In summary, the main contributions of this paper are:

- A simulated training environment
- A neural network architecture for separation

- A deep reinforcement learning agent to train the network based on experiences gained in the environment
- An evaluation of the new deep reinforcement learning method
- A comparison between the supervised learning and the reinforcement learning approach



Fig. 1: Entangled grasping situation in the simulated training environment

2. State of the Art

Most of today's automated bin picking systems for randomly located industrial parts do not address the problem of entangled workpieces [1] [2] [5]. Nevertheless, motion planning in random bin picking can rise in complexity by entanglements caused by complex shapes of workpieces. Finding a collision-free trajectory after gripping an object, as shown in Ellekilde et al. [6], Iversen et al. [7], and Spenrath et al. [8], can fail due to the other workpieces, which are still entangled during the extraction of the gripped object. To avoid that, Moosmann et al. [3] focused on a model-based approach to detect those situations and furthermore on a supervised learning approach to systematically separate entangled workpiece situations in a pile in random bin picking with six degrees of freedom motion planning [4]. A method to detect gripping points that can lead to a failed grasp is proposed by Matsumura et al. [9]. Another approach to detect entangled workpiece situations and separate them is presented by Leao et al. [10], which is limited to only two dimensional solving strategies and simple rebuilding of entanglement shapes. Even though the supervised learning approach [4] can disentangle a broad range of complex workpieces, it requires significant amounts of training data and time.

In contrast to supervised learning, reinforcement learning might not require a large amount of labeled training data [11]. Producing fully autonomous agents that interact with their environments to learn optimal behaviors, improving over time through trial and error is one of the primary goals in the field of reinforcement learning [11]. One of the long-standing challenges is learning to control the agent from high-dimensional sensory inputs like vision and speech. Solving physics-based control problems [12], object recognition with visual attention [13] and high-dimensional robot control [14] are only a few examples of the wide range, where deep reinforcement learning has proved mastering human-level control policies successfully. The use of convolutional neural networks is a popular technique for learning features and visual models with raw image data and complex reinforcement learning environments [15] [16]. Especially Deep Q-Networks have shown to be effective in playing Atari 2600 games and therefore proven to master difficult control policies using only raw pixel data [15] [17]. Reinforcement learning, which is based on the experiences made in the training environment and the associated rewards, is employed throughout this work. The reward definition is one of the main challenges in deep reinforcement learning and a focus topic in this paper.

3. Method Overview

The main components of the reinforcement learning method for separating entangled workpieces and their interactions are shown in Fig. 2.

At runtime, the policy network (Sec. 5) is presented with sensor inputs (state) from which it predicts an adequate separation trajectory according to the separation motion model (Sec. 4). This is represented by the testing section on the upper left side bounded by the dotted line.

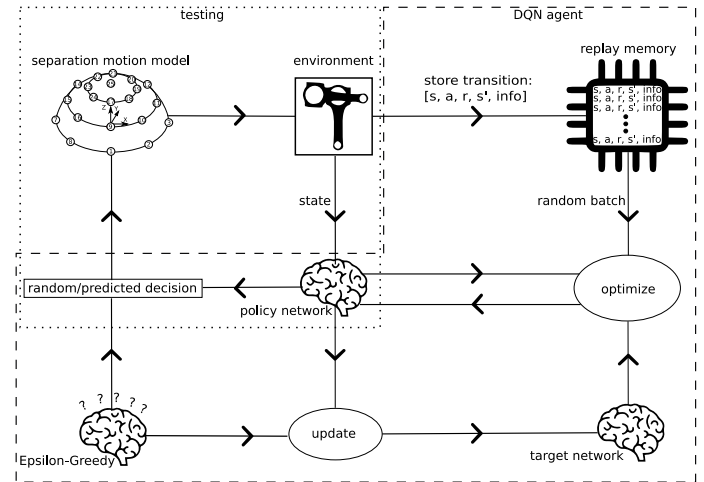


Fig. 2: Method big picture and structure of interactions

The DQN agent (Sec. 6.1) trains the policy network on the desired behaviors using a simulation environment (Sec. 4.1). Various simulation experiences are stored as transitions in the replay memory (Sec. 6.2), which is used to optimize the policy network. The associated target network is then updated by the policy network. Random decisions of optimal actions in training according to the Epsilon Greedy strategy ensure exploration of optimal actions. This procedure is bounded by the dashed line.

The interaction of the DQN agent with the simulation environment and the neural network is described in more detail in Sec. 6.3.

4. Separation Motion Model

The separation method is based on three hemispheres, as described by Moosmann et al. [4]. Each hemisphere consists of 25 points surrounding the origin of the gripped workpiece, as shown in Fig. 3. The origins of the second and third hemisphere are variable and depend on the previously chosen hemisphere points. This yields 15,625 possible combinations of points from the different hemispheres and therefore a lot of possible trajectories for moving the workpiece in order to solve entanglements. The orientation of the hemisphere and thus the exact location of the 25 points are set relatively to the orientation of the gripped workpiece.

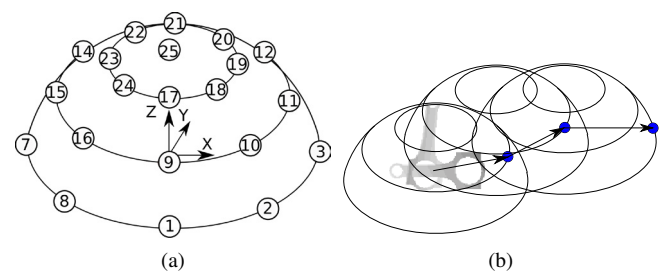


Fig. 3: a) 25 possible path points of a hemisphere; b) Possible trajectory with three hemispheres [4]

4.1. Simulation Environment

The entanglement and the separation process are simulated in CoppeliaSim with the toolkit PyRep in order to have a simple and flexible API and a significant speedup in run-time [18]. A cropped depth image as input for the neural network is created by a vision sensor in the simulation scene [3]. The simulation setup consists of two entangled workpieces, which are moved to the possible separation points on the hemisphere. The gripping points and the collision-free trajectory for gripping the workpiece are calculated by the $bp3^{TM}$ software developed by Fraunhofer IPA. The separation process is used only if an entanglement is identified. The identification of entangled situations is done by Moosmann et al. [3]. The exact calculation of the hemisphere points is described by Moosmann et al. [4].

4.2. Workpiece Geometries

To evaluate the algorithm, three different object geometries are used: connecting rods, round steel brackets and hooks. All of them create unique entanglements for their geometries and therefore different separation strategies are needed. A representative example of an entanglement for each shape is shown in Fig. 4. Obviously the complexity of separating the hooks is higher than for example the complexity for the connecting rods. This creates the possibility to prove, whether the approach is applicable in a wide range of different workpiece geometries or not.

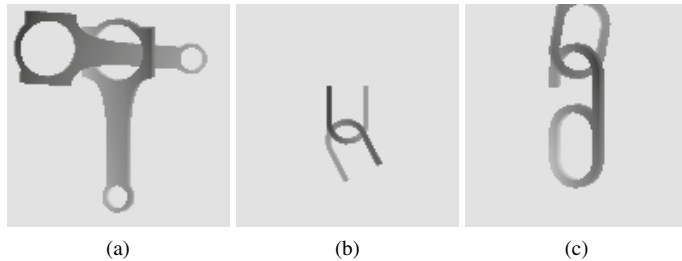


Fig. 4: Example of a possible entanglement for different workpiece geometries. a) Connecting rod; b) Round steel bracket; c) Hook

5. Network Structure

A convolutional neural network, as shown in Fig. 5, is used for the image classification and feature abstraction. The input data of the neural network consists of a depth image with the size of 128×128 pixels. The structure of the convolutional neural network is based on the neural network used in Mnih et al. [19], which leads to impressive solutions in Atari gameplay. Furthermore we added two max-pooling layers to the convolutional neural network to decrease the number of inputs for the fully connected layer from 9,216 to 64. The fully connected layer of 512 neurons finally chooses an action right before the output layer of 25 neurons for the 25 possible path points. The output of these 25 neurons contains a value how promising an action leads to a separation. Every convolutional layer and the

fully connected layer use a rectified linear unit (ReLU) as activation function.

RMSProp is used as optimizer, which is a mini-batch learning version of resilient back-propagation (RProp) [20]. RMSProp normalizes the gradient by using a moving average of squared gradients. This optimizer is the same as used in Mnih et al. [19].

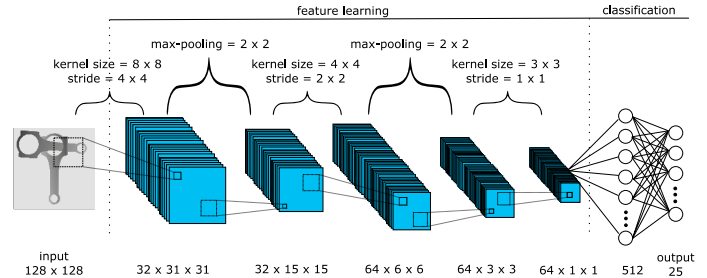


Fig. 5: Architecture of the convolutional neural network

6. Deep Reinforcement Learning Agent

Reinforcement learning decides which action will yield the best reward by trial and error. Mostly, the actions affect not only the subsequent reward but also the next action. This is the reason why it is affecting all subsequent rewards [21]. Trial and error search and the delayed reward build the two most prominent features of reinforcement learning [22]. Reinforcement learning agents can be classified by several characteristics for example, whether it is a model-based or model-free method, a policy-based or a value-based method or whether it is an on- or an off-policy learning method. One main task is to choose a method and an agent, which is able to solve the addressed problem in a stable and a quick way. Hence the identification of the problem characteristics is essential. The separation task can be characterized as a discrete action-space problem, due to the given number of actions limited to 25 action values on each hemisphere. In addition, the discovery of the environment has to be part of the agent, because no exact model of the environment is available. An overview of the taxonomy of the most prominent (deep) reinforcement learning agents is shown by Dong et al. [23].

6.1. DQN Agent

The Deep-Q Network (DQN) as a model-free, value-based, off-policy method solves all the described problems of this separation task and shows a stable solution for smaller networks like it is used in this paper. Moreover, value-based methods usually converge faster than policy-based methods and have a higher sample efficiency [21]. Another reason for choosing the DQN agent is the Epsilon-Greedy strategy, which deals with the trade-off between exploration and exploitation [21]. Thanks to the Epsilon-Greedy strategy the agent does not only take the chosen actions by the policy into account but it also takes random actions for exploration with the probability ϵ and uses these experiences to train the policy as well. This prevents the

agent from stagnation in a local optimum. The achieved results playing Atari games with DQN agents prove the effective usage for training a convolutional neural network with raw pixel data as input [15]. Typical for off-policy learning algorithms is the Temporal Difference (TD) method, which yields to a smaller variance than the Monte Carlo (MC) method, which is used in basic policy based algorithms. TD as a compromise between Dynamic Programming (DP) and MC uses bootstrapping for estimation but in contrast to DP it does not require the full knowledge of the environment [21]. One main difference between MC and TD is how the learning update is done. While MC has to wait until the end of an episode to update, TD can update at each time step [23]. The TD method for updating the action-value function $Q(s, a)$ for a Q-learning method is shown in Equation 1, where s describes the current state, a the action, α the learning rate, γ the discount factor, r the reward and $\max_{a'}(Q(s', a'))$ the estimation of the maximal future Q-value [21].

$$Q(s, a) \leftarrow (1 - \alpha)Q(s, a) + \alpha[r + \gamma \max_{a'}(Q(s', a'))] \quad (1)$$

6.2. Replay Memory

We use experience replay [24] and store the agent's experience at each time step over many episodes in a replay memory. The experiences stored consist of the actual state s , the chosen action a , the reward r , the next state s' , which will be reached by taking the chosen action in the actual state and an information *info* whether the state is the last one of the actual episode or not. This information is important for the TD algorithm, in which the value of $Q(s', a')$ becomes 0 if the current state is already the last state in the episode where an action can be chosen. During mini-batch updates, a random pool of stored samples of the replay memory is used to perform Q-learning. Thanks to the fact, that each piece of experience is potentially used in a lot of weight updates, the data efficiency increases. Using experience replay avoids oscillations or divergence in the parameters [15].

6.3. Structure of Interaction between DQN Agent and the Environment

Fig. 2 demonstrates the DQN agent's interaction with the environment and the optimization of the policy network with the usage of the replay memory. The dashed line shows the DQN agent and its components. The DQN agent has two input lines from the environment, the state, which goes directly to the policy network and the transition of state, action, reward, next state and *info*, which is connected directly to the replay memory. As output from the DQN agent the chosen action on the upper left side can be seen, which is the only input data for the environment. A detailed look at the DQN agent shows, that the action can be chosen at the random/predicted decision. The input for this decision comes from the policy network based on the state value, or from the Epsilon-Greedy strategy, which takes random actions independent from the current state. If an action has been chosen and the environment moves from state s to the next state s' the transition s, a, r, s', info is saved in the replay memory, while the next state s' becomes the new state s

and builds the next input data for the policy network. In order to train the policy network, the stored transitions in the replay memory are used as random batches with the batch-size of 64 transitions. From these transitions a loss function is calculated, which depends on the Q-value of the state s calculated by the policy network, and the Q-value of the next state s' calculated by the target network. The target network is an exact copy of the policy network, but it is not optimized during the learning process, in order to overcome the effects of oscillation or divergence and to increase stability [19]. The target network is updated every 1,000 learning steps with the policy network. The loss function is used for optimizing the policy network with back-propagation (RMSProp).

If the network is trained and the solution converges, the testing can start. For testing, which does not optimize the policy network anymore, fewer components are needed. The relevant elements for testing are bounded by the dotted line and only consist of the policy network, which chooses an action without the opportunity of a random action chosen by the Epsilon-Greedy strategy and the environment. The state is taken into account as input to the network, while the environment still only gets the chosen action as input.

6.4. Reward Shaping

The reward as the main component and quality measure in reinforcement learning has to be representative and should guide the agent to the right decision. Therefore it is necessary to implement a reward that is not only depended on solving the problem but also gives the agent an imagination how far away from separation it is. To qualify the separation process the Euclidean distance is used in this paper. There are two main components of the reward taken into account. First component r_1 is based on the Euclidean distance between the position of the gripped workpiece g and the entangled workpiece e to give a value of separation as shown in Equation 2.

The second component describes the distance the entangled workpiece was moved from its initial position i due to the movement of the gripped workpiece as shown in Equation 3. The second component is set to a negative value, because the best separation should move the other workpiece, which was not gripped, as little as possible.

In order to get the best solution these components have to be weighted and summed up as shown in Equation 4.

$$r_1 = \sqrt{(g_x - e_x)^2 + (g_y - e_y)^2 + (g_z - e_z)^2} \quad (2)$$

$$r_2 = -\sqrt{(i_x - e_x)^2 + (i_y - e_y)^2 + (i_z - e_z)^2} \quad (3)$$

$$r = \lambda_1 \cdot r_1 + \lambda_2 \cdot r_2 \quad (4)$$

While trying to find the optimal values for λ_1 and λ_2 to deal with the separation process the physical dependence of both components was clearly visible, so that a high movement of the entangled workpiece was acceptable, if the distance was long enough. If λ_1 is much higher than λ_2 , the agent learns to throw the entangled workpiece as far as possible. On the other side, if

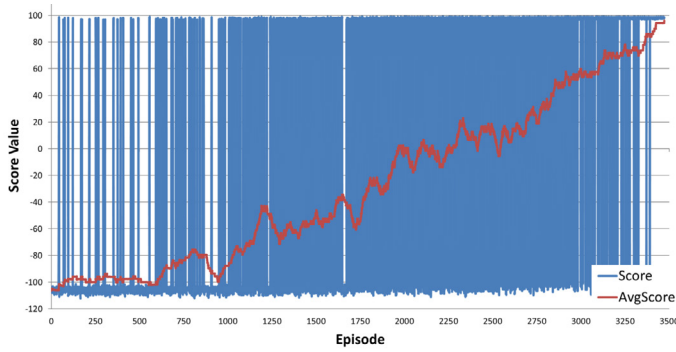


Fig. 6: Score and average score of connecting rods separation environment

λ_2 is significantly higher, the agent learns to move in the hemispheres without touching or moving the entangled workpiece. The agent does not try to separate at all in this case, so that at the end of the chosen actions the result was nearly the same as the initial position of both workpieces. Moreover in complex shapes, like the hooks, the separated solution can be reached due to a movement of the gripped workpiece close to the origin of the entangled workpiece. The maximum possible distance in which the hooks can still be entangled while the entangled workpiece did not move can be even larger than the required separation distance. This leads to a worse reward for separations than for some entangled situations. So these components alone do not give any feedback about a good separation, therefore a better verification of the separation is needed.

That is the reason why in the following r_1 does not affect the final reward anymore, but at the end of each episode the gripped workpiece is lifted in the air. If the position of the entangled workpiece does not change during the lifting of the gripped object at the end of an episode a positive reward c_s is added. Otherwise the object is still entangled and the reward c_s is subtracted. So finally the reward can be defined as shown in Equation 5. This yields to a separation with a minimal movement of the entangled workpiece. The value of c_s is set to 100 after moving to a possible path point in the third hemisphere.

$$r = \begin{cases} \lambda_2 \cdot r_2 + c_s & ;\text{if separated} \\ \lambda_2 \cdot r_2 - c_s & ;\text{if not separated} \end{cases} \quad (5)$$

7. Results

In this chapter the solutions of the separation method used for the three geometries described in Section 4.2 are visualized and analyzed. The simulations for all entanglements were done with the following parameters.

- Radius of each hemisphere = 0.035 m
- Learning rate = 0.003
- Batch-size = 64
- $\epsilon_{start} = 1.0$; $\epsilon_{end} = 0.001$; $\epsilon_{step} = 0.0001$

7.1. Validation of Separation Strategy

Our deep reinforcement learning separation strategy proves that this network structure and the DQN agent with the created environment can learn to separate different geometries with dif-

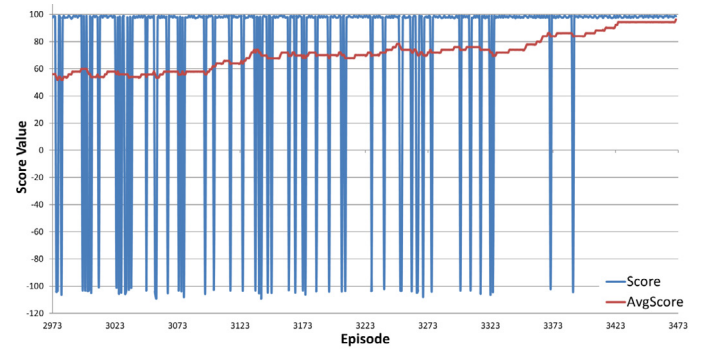


Fig. 7: Score and average score of the last 500 episodes of connecting rods separation environment

ferent entanglements. The loss function is not converging as fast as separation strategies are found in a stable way. This depends on the huge amount of possible separations in the 15,625 possible trajectories. Therefore the convergence was detected by the average score defined by the mean value of the last 100 saved scores. The score is calculated by the sum of each episode reward.

In Fig. 6 the score and the average score of the separation of connecting rods over the number of episodes is shown. The number of entangled situations from episode 0 to 1,000 is much higher than between the episodes 1,000 to 3,000, where the average score is rising. The reason for this are the randomly chosen actions based on Epsilon-Greedy strategy and also the training of the neural network. To see the effect of convergence, the last 500 episodes are shown in Fig. 7. During the last 100 episodes the separation failed only once, which depends on the Epsilon-Greedy strategy, and in the last 60 episodes every separation process was successful. The simulation converged in around 3,500 episodes for the connecting rods, the round steel brackets and the hooks as well. The diagrams of the round steel brackets and the hooks look quite similar to the ones, which are shown for the connecting rods. Even very complex entanglements have been solved with this separation technique. One very difficult situation is shown in Fig. 8.

7.2. Comparison with Supervised Learning Approach

To compare the reinforcement learning approach with the supervised learning approach shown in Moosmann et al. [4], the focus is on the following parameters:

- Training time
- Success rate
- Complexity of possible entanglements
- Complexity of neural network structure

As shown in Fig. 9 the reinforcement learning approach reaches an efficiency of 100 percent for separating the testing data in less than 10 hours. On the other side the supervised learning approach requires more than 300 hours to reach the same success rate. The time, which is taken into account here, is not only the time the neural networks need to converge, but also the time which is needed to produce the training data. Therefore

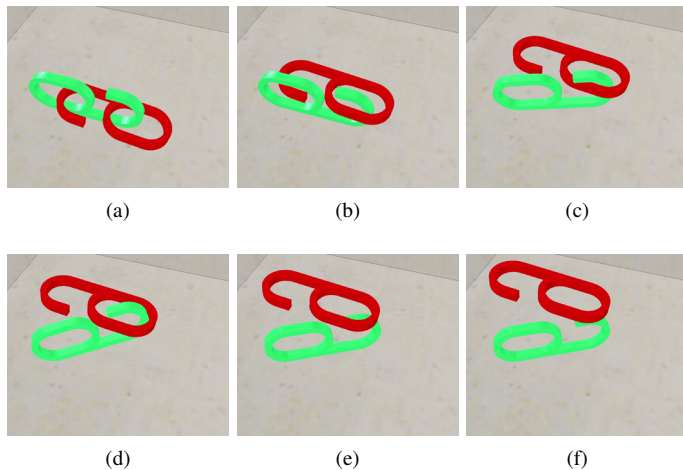


Fig. 8: Separation of a difficult hook entanglement. a) initial position; b) position after moving to the point of the first hemisphere; c) position after moving to the point of the second hemisphere; d) position at critical entanglement point; e) position after moving to the point of the third hemisphere; f) position during lifting

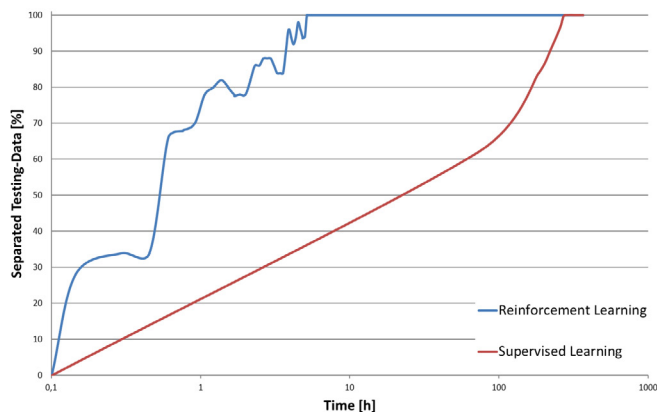


Fig. 9: Comparison of the efficiency between supervised and reinforcement learning approach

the reinforcement learning approach shows a 30 times better efficiency. Furthermore as it is shown in Fig. 8 the reinforcement learning approach is able to identify separation methods for complex entanglements. The DenseNet used for the supervised learning approach [4] is even deeper than the convolutional neural network shown in Fig. 5, which is used for the reinforcement learning approach. The time needed to predict an action is less than 4 ms using a CPU with 16 GB RAM.

The reinforcement learning approach described in this paper outperforms the supervised learning approach in every single point of comparison.

8. Conclusion and Outlook

In this paper, we proposed a new deep reinforcement learning method for separating entangled workpiece situations in random bin picking. We introduced a simulation environment for training reinforcement learning agents and set up a network structure for learning from raw pixel data. The selection of an effective deep reinforcement learning agent based on the separation task characterizations was achieved. Further-

more we demonstrated the interaction between environment and the agent in detail. We have shown that the deep reinforcement learning agent is an attractive alternative to the supervised learning approach presented by Moosmann et al. [4] and yields to a separation strategy even faster. In summary this approach copes with entanglements of different workpiece shapes in impressively short time. In future work highly cluttered scenes with more than two entangled workpieces will be implemented. If the separation method is trained on enough simulated data, the transfer from simulation to reality will be addressed like it is described by in Moosmann et al. [3].

References

- [1] Spenrath, F., Pott, A.: Using Neural Networks for Heuristic Grasp Planning in Random Bin Picking. In: IEEE, 2018.
- [2] Oh, J. K., Lee, S., Lee, C. H.: Stereo vision based automation for a bin-picking solution. IJCAS, 2012.
- [3] Moosmann, M., Spenrath, F., Kleeberger, K., Khalid, M.U., Mönnig, M., Rosport, J., Bormann, R.: Increasing the Robustness of Random Bin Picking by Avoiding Grasps of Entangled Workpieces. In: CIRP CMS, 2020.
- [4] Moosmann, M., Spenrath, F., Mönnig, M., Khalid, M.U., Jaumann M., Rosport, J., Bormann, R.: Using Deep Neural Networks to Separate Entangled Workpieces in Random Bin Picking. In: Proceedings of SCAP, 2020.
- [5] Martinez, C., Boca, R., Zhang, B., Chen, H., Nidamarthi, S.: Automated bin picking system for randomly located industrial parts. In: IEEE 2015.
- [6] Ellekilde, L.P., Petersen, H. G.: Motion planning efficient trajectories for industrial bin-picking. In: IJRR, p. 991-1004, 2013.
- [7] Iversen T. F., Ellekilde L. P.: Benchmarking motion planning algorithms for bin-picking applications.. In: Industrial Robot, p. 189-197, 2017.
- [8] Spenrath F., Spiller A., Verl A.: Gripping point determination and collision prevention in a bin-picking application. ROBOTIK, 2012.
- [9] Matsumura R., Harada K., Domae Y., Wan W: Learning Based Robotic Bin-picking for Potentially Tangled Objects. In: IEEE/RSJ IROS, 2019.
- [10] Leao G., Costa C. M., Sousa A., Veiga G.: Detecting and Solving Tube Entanglement in Bin Picking Operations. In: Applied Sciences, 2020.
- [11] Arulkumaran K., Deisenroth M. P., Brundage M., Bharath A. A.: A Brief Survey of Deep Reinforcement Learning. In: arXiv, 2017.
- [12] Heess, N.; Wayne, G.; Silver, D.; Lillicrap, T.; Erez, T.; and Tassa, Y.: Learning continuous control policies by stochastic value gradients. In: Advances in NIPS, 2944–2952, 2015.
- [13] Mnih V., Heess N., Graves A., Kavukcuoglu K.: Recurrent Models of Visual Attention. In: NIPS, 2014.
- [14] Levine S., Finn C., Darrell T., Abbeel P.: End-to-end training of deep visuomotor policies. In: Journal of ML Research 17(39):1–40, 2016.
- [15] Mnih V., Kavukcuoglu K., Silver D., Graves A., Antonoglou I., Wierstra D., Riedmiller M.: Playing Atari with deep reinforcement learning. In: arXiv, 2013.
- [16] Lenz I., Lee H., Saxena A.: Deep Learning for Detecting Robotic Grasps. In: arXiv, 2014.
- [17] Hausknecht M., Stone P.: Deep Recurrent Q-Learning for Partially Observable MDPs. In: arXiv, 2017.
- [18] James S., Freese M., Davison A.J.: PyRep: Bringing V-REP to Deep Robot Learning. In: arXiv, 2019.
- [19] Mnih V. et al.: Human-level control through deep reinforcement learning. In: Nature, 518 (7540): 529 - 533, 2015.
- [20] Hinton G.: Lecture 6.5: Divide the gradient by a running average of its recent magnitude. In: COURSE: NN for ML, 2012.
- [21] Sutton R. S., Barto A. G.: Reinforcement learning: An introduction. In: MIT press, 2018.
- [22] Joshi S., Kumra S., Sahin F.: Robotic Grasping using Deep Reinforcement Learning. In: arXiv, 2020.
- [23] Dong H., Ding Z., Zhang S.: Deep Reinforcement Learning Fundamentals, Research and Applications. In: Springer Nature, 2020.
- [24] Lin L.: Reinforcement learning for robots using neural networks. In: Technical report, DTIC Document, 2012.